Celebal Summer Internship 2026

Week 4 Task: Azure Cloud Fundamentals and Data Pipeline Implementation using ADF

By : Shaik Faizan Ahmed

Roll No : 23B81A05L3

College : CVR COLLEGE OF ENGINEERING

Task 1: Azure Portal Exploration and Resource Group Creation

Objective

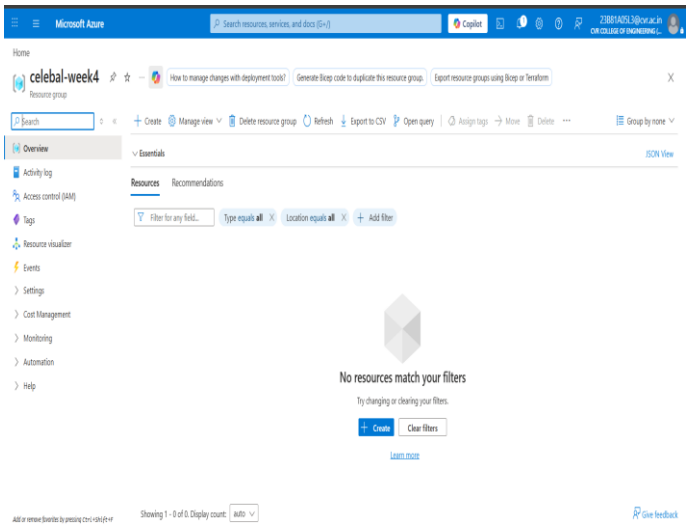
To explore the Azure Portal and create a Resource Group for organizing Azure resources.

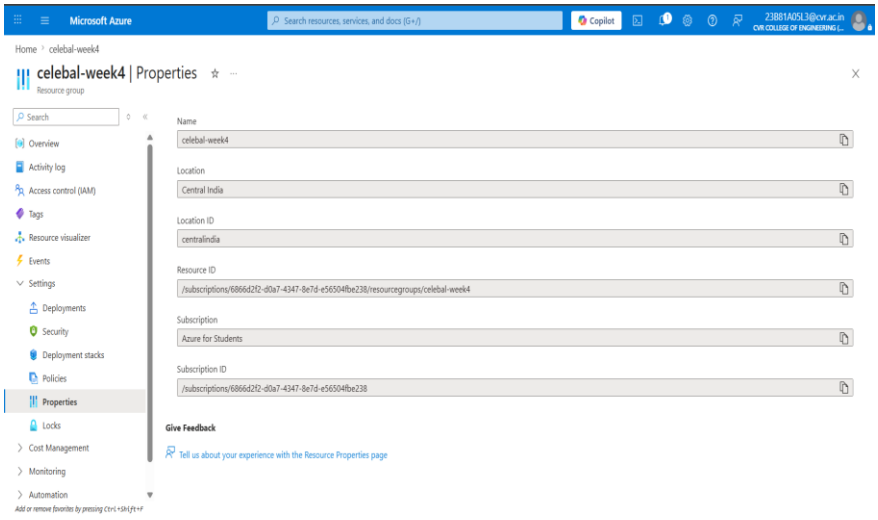
Steps Performed

1. Logged into the Azure Portal using the Azure for Students account.
2. Searched for Resource Groups in the Azure Portal.
3. Clicked on Create Resource Group.
4. Selected the Azure subscription.
5. Entered the Resource Group name as celebal-week4.
6. Selected the region as Central India.
7. Clicked Review + Create and then Create.
8. Verified that the Resource Group was successfully created.

Output

A Resource Group named celebal-week4 was successfully created and used to manage all resources required for the assignment.





What I Learned

I learned how Azure resources are organized using Resource Groups. I understood that a Resource Group acts as a logical container that helps manage and organize related cloud resources in a single place.

Task 2: Storage Setup

Objective

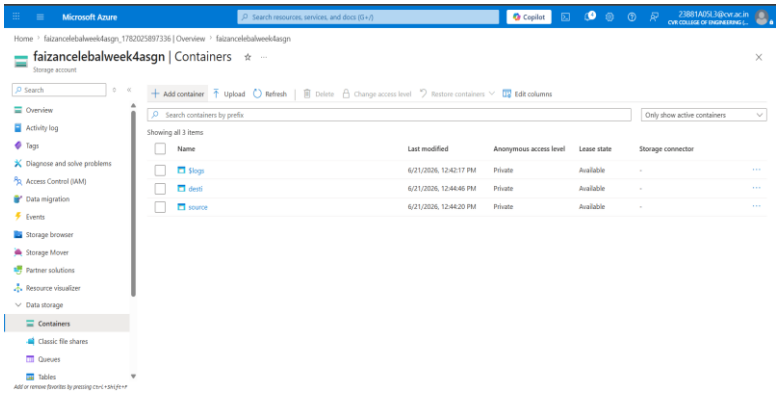
To create an Azure Storage Account, create Blob Storage containers, and upload CSV files.

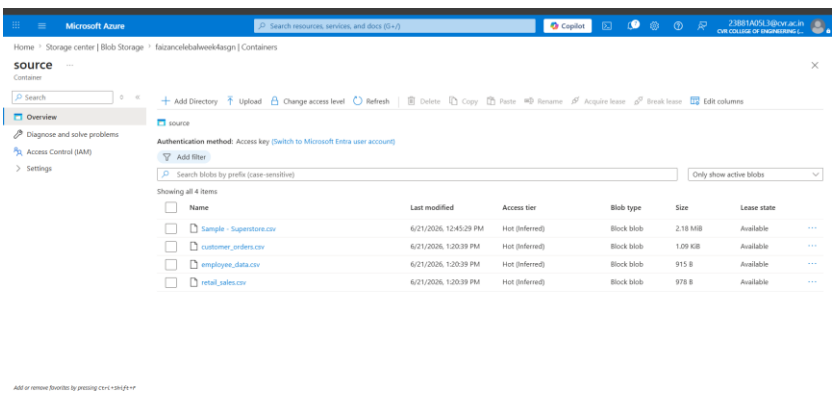
Steps Performed

1. Opened Storage Accounts in Azure Portal.
2. Created a new Storage Account named faizancelebalweek4asgn.
3. Selected the existing Resource Group.
4. Configured the storage settings and completed the creation process.
5. Opened Blob Storage and created two containers:
    - source
    - desti
6. Uploaded the following CSV files into the source container:
    - Sample-Superstore.csv
    - retail_sales.csv
    - employee_data.csv
    - customer_orders.csv
7. Verified that all files were uploaded successfully.

Output

The Storage Account and Blob Containers were successfully created, and all CSV files were uploaded into the source container.





What I Learned

I learned how Azure Blob Storage is used to store files in the cloud. I understood how containers are used to organize files and how data can be uploaded and managed using Azure Storage.

Task 3: Azure Data Factory Basics

Objective

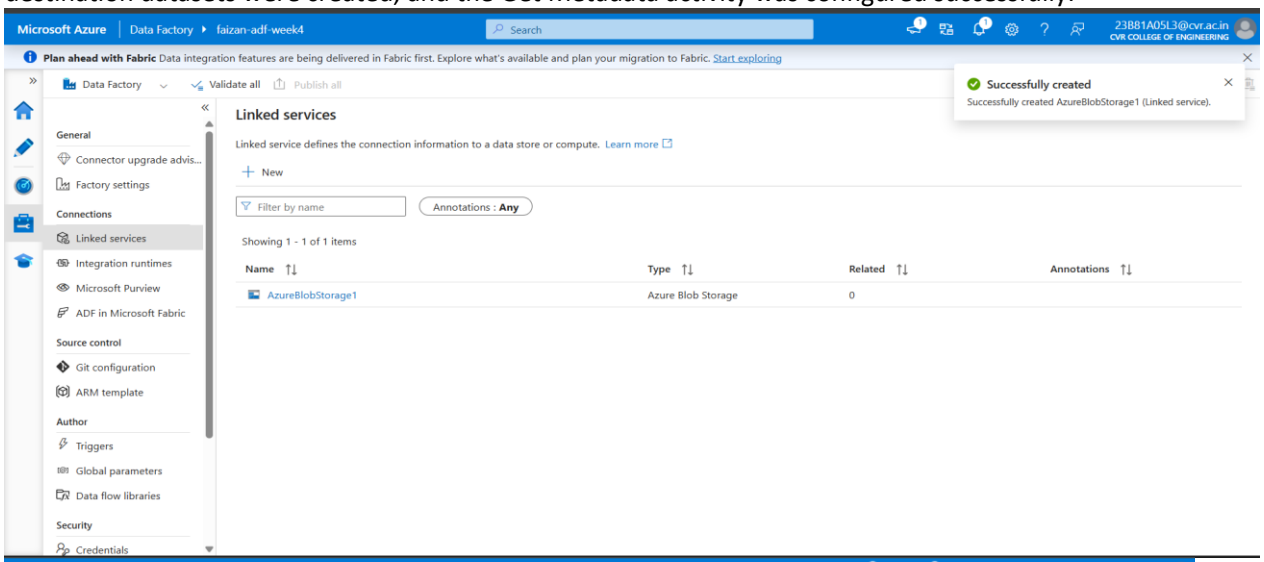
To create Azure Data Factory and configure the required components for data movement.

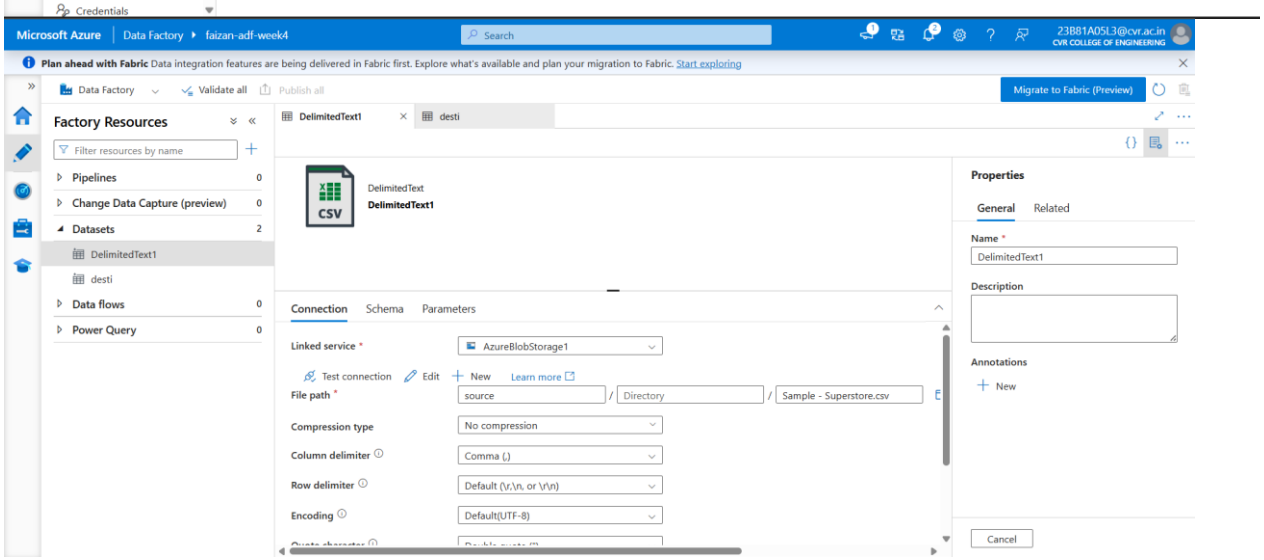
Steps Performed

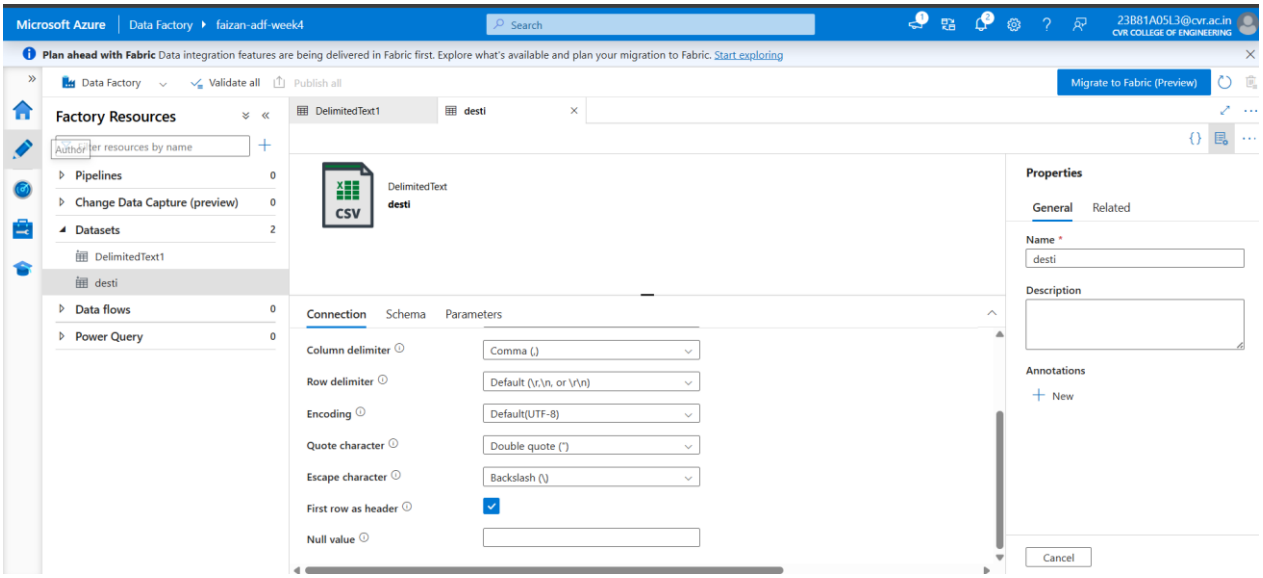
1. Created Azure Data Factory in the Resource Group.
2. Opened Azure Data Factory Studio.
3. Explored the Author, Monitor, and Manage sections.
4. Created a Linked Service to connect Azure Data Factory with Azure Blob Storage.
5. Created datasets for source and destination storage locations.
6. Configured a Get Metadata activity.
7. Configured the activity to retrieve Child Items from the source container.

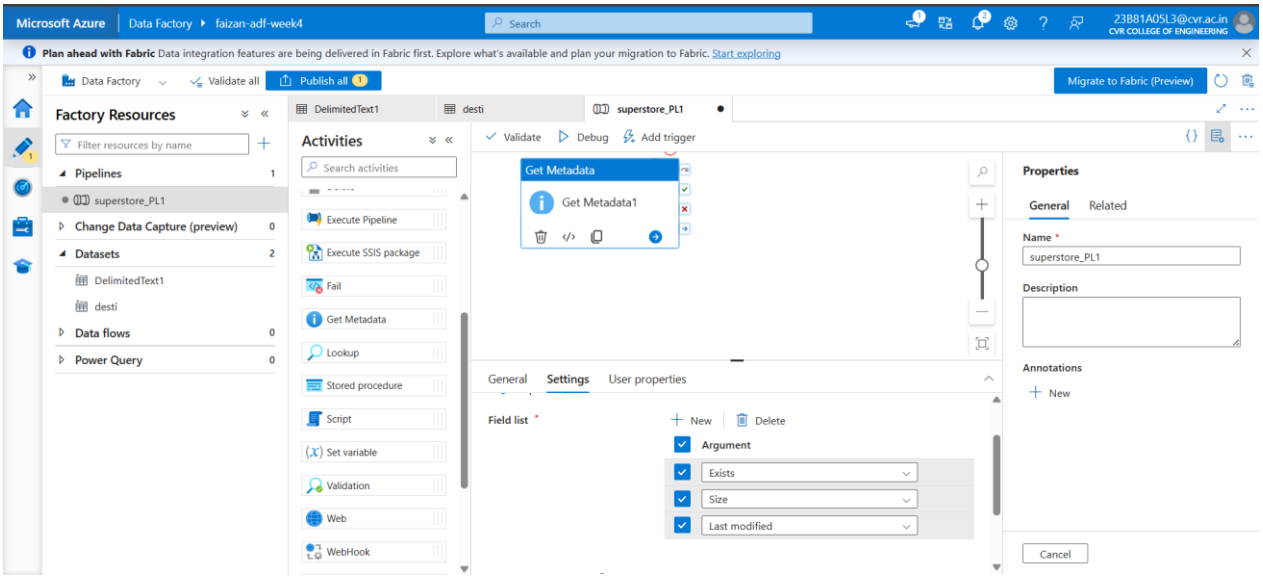
Output

Azure Data Factory was successfully connected to Azure Blob Storage using a Linked Service. Source and destination datasets were created, and the Get Metadata activity was configured successfully.









What I Learned

I learned how Azure Data Factory connects to external storage systems using Linked Services. I also learned how datasets represent data sources and how the Get Metadata activity can be used to retrieve information about files before processing them.

Task 4: Pipeline Development

Objective

To develop an Azure Data Factory pipeline that retrieves file information from Blob Storage and copies multiple files from the source container to the destination container.

Steps Performed

1. Opened Azure Data Factory Studio and navigated to the Author section.
2. Created a new pipeline named superstore_PL1.
3. Added a Get Metadata activity to retrieve the list of files available in the source container.

4. Configured the Get Metadata activity to use the DS_SourceFolder dataset.
5. Selected Child Items in the Field List to retrieve all files present in the source container.
6. Added a ForEach activity to iterate through all files returned by the Get Metadata activity.
7. Configured the ForEach activity using the expression:

@activity('Get_File_Metadata').output.childItems

8. Added a Copy Data activity inside the ForEach activity.
9. Configured the source dataset to dynamically read each file using the FileName parameter.
10. Configured the destination dataset (desti) to dynamically create the corresponding file in the processed container.
11. Connected the activities in the following sequence:

Get Metadata → ForEach → Copy Data

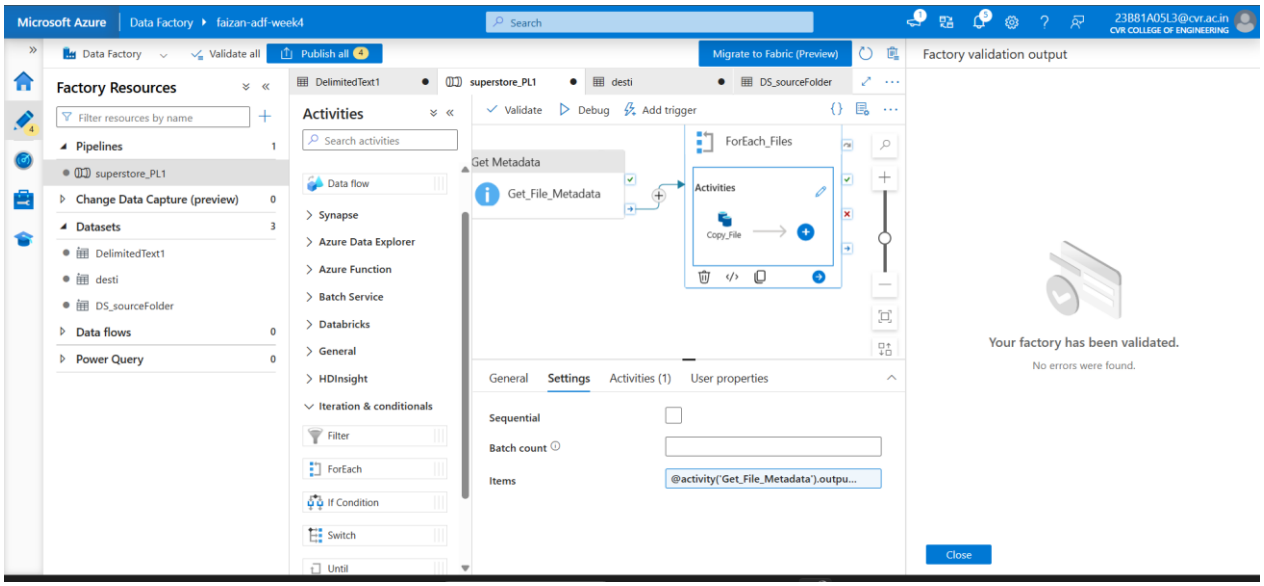
12. Validated the pipeline and confirmed that no errors were found.
13. Published all changes successfully.

Output

A dynamic Azure Data Factory pipeline was successfully developed. The pipeline retrieves all files from the source container, processes each file using the ForEach activity, and copies them to the destination container automatically.

Pipeline Workflow:

Get Metadata → ForEach → Copy Data



What I Learned

I learned how to design and build data pipelines in Azure Data Factory. I understood how activities can be connected to create workflows and how the ForEach activity can be used to process multiple files automatically. I also learned how dataset parameters enable dynamic file processing, making the pipeline scalable and reusable.

Task 5: Pipeline Execution

Objective

To execute the Azure Data Factory pipeline and monitor its execution status.

Steps Performed

1. Opened the developed pipeline in Azure Data Factory Studio.
2. Clicked on Validate to ensure the pipeline configuration was correct.
3. Confirmed that no validation errors were present.
4. Published all pipeline changes using the Publish All option.
5. Executed the pipeline using the Debug option.
6. Opened the Monitor section to track pipeline execution.
7. Verified the status of all activities within the pipeline.
8. Confirmed that Get Metadata, ForEach, and Copy Data activities completed successfully.
9. Checked the destination container to verify that all files were copied successfully.

Output

The pipeline executed successfully without any errors.

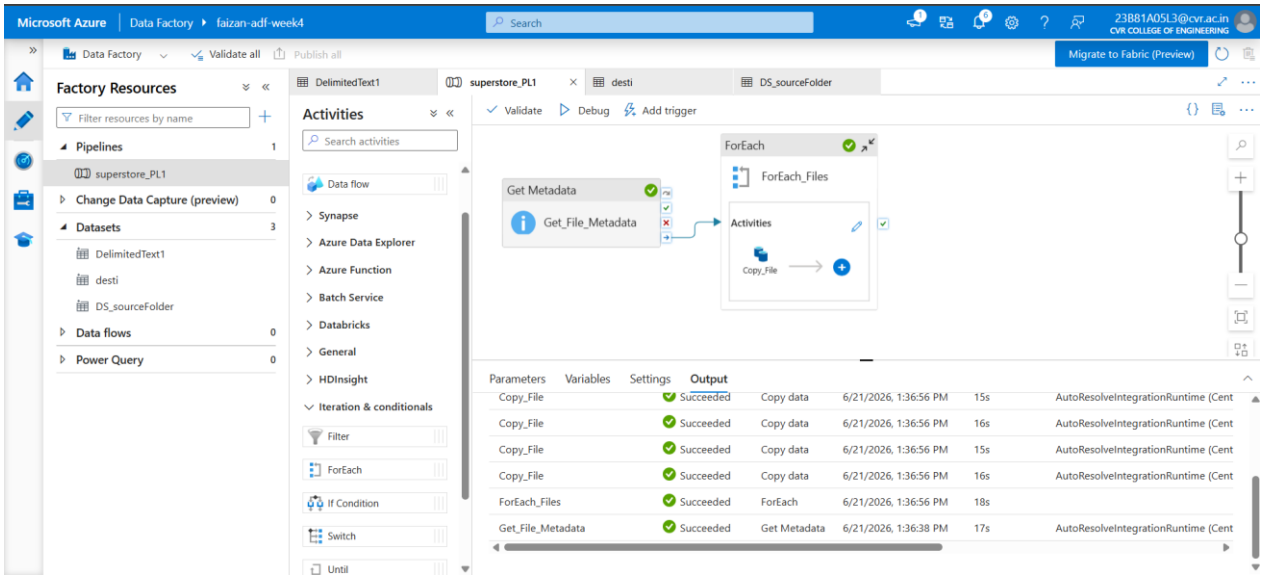
Execution Results:

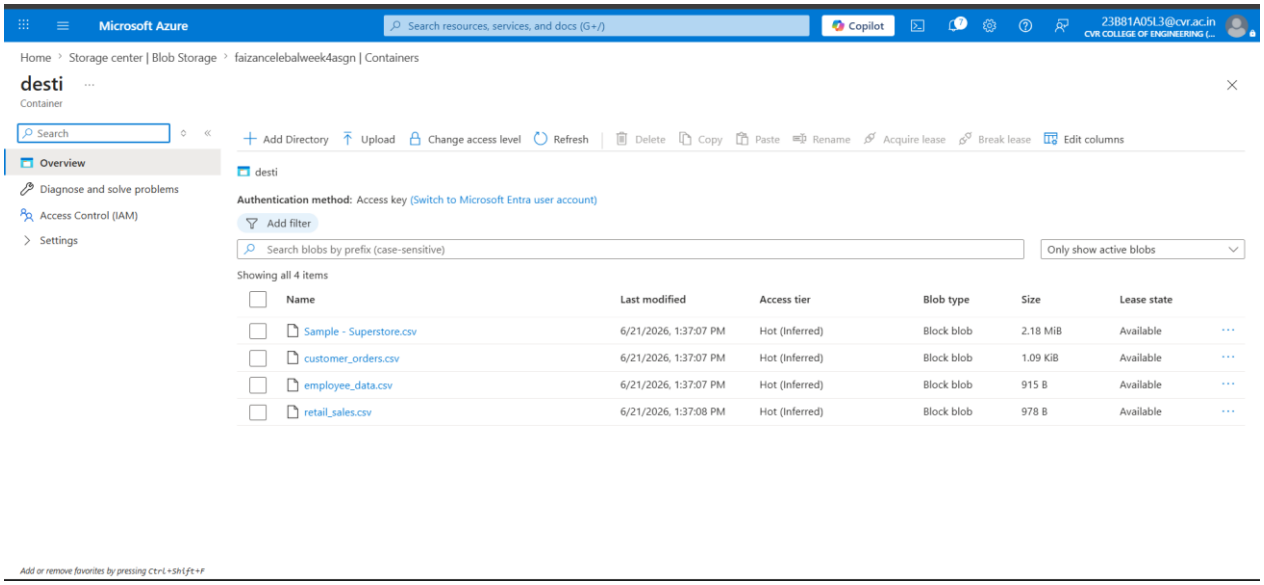
- Get Metadata Activity: Succeeded
- ForEach Activity: Succeeded
- Copy Data Activity: Succeeded
- Total Files Processed: 4
- Total Files Copied: 4

Files Processed:

- Sample-Superstore.csv
- retail_sales.csv
- employee_data.csv
- customer_orders.csv

The successful execution confirmed that the pipeline was able to retrieve metadata, iterate through multiple files, and copy them from the source container to the destination container.





What I Learned

I learned how to execute and monitor pipelines in Azure Data Factory. I understood how to validate pipeline configurations before execution and how to monitor activity status using the Monitor section. I also learned how to verify successful data movement by checking the destination storage location after pipeline execution.

Task 6: IAM Roles and Access Management

Objective

To understand Azure Identity and Access Management (IAM) and configure the required permissions for accessing Azure Storage resources.

Steps Performed

1. Opened the Azure Storage Account.
2. Navigated to Access Control (IAM).
3. Reviewed the existing role assignments for the Storage Account.
4. Assigned the Reader role to provide read-only access to storage resources.
5. Configured Storage Blob Data Contributor permissions to allow data access and file operations within Blob Storage.
6. Verified that the required permissions were successfully assigned.
7. Confirmed that Azure Data Factory had the necessary access to interact with Azure Blob Storage during pipeline execution.

Roles Configured

Reader

The Reader role was assigned to provide permission to view Azure resources and configurations without making modifications.

Storage Blob Data Contributor

The Storage Blob Data Contributor role was configured to allow access to Blob Storage data, including reading, writing, and managing files within containers.

Owner (Existing Permission)

The account already had Owner permissions, which provide full management access to Azure resources. Therefore, assigning the Contributor role separately was not required.

(As per the assignment requirements, the Contributor role was also intended to be assigned. However, while configuring IAM permissions, the Contributor role was not available for assignment under the current Azure for Students subscription and access configuration. During the Saturday mentor meeting, this issue was discussed with the mentors, and they advised us to proceed with the available permissions and leave the Contributor role assignment.)

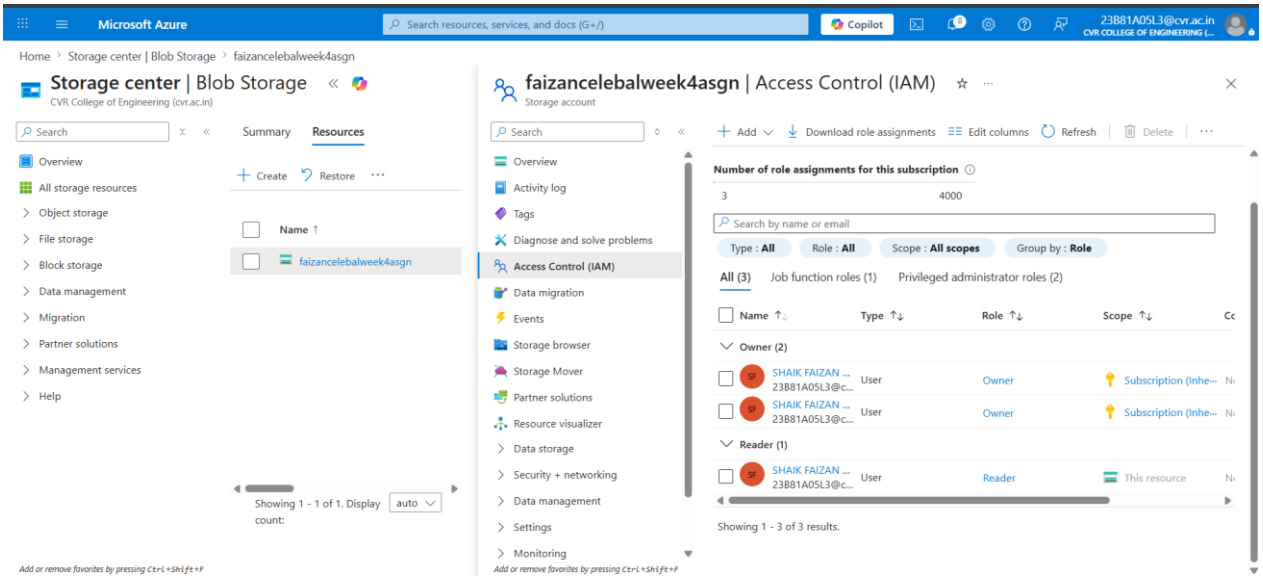
Output

The required IAM permissions were successfully configured, ensuring secure access between Azure Data Factory and Azure Blob Storage.

Configured Roles:

- Owner
- Reader
- Storage Blob Data Contributor

These permissions enabled successful execution of the data pipeline and file transfer operations.



What I Learned

I learned how Azure Identity and Access Management (IAM) is used to control access to cloud resources. I understood the purpose of different roles such as Reader, Storage Blob Data Contributor, and Owner. I also learned that proper permissions are essential for enabling Azure services to securely access and process data stored in Azure Storage.

MINI PROJECT REPORT

Title

End-to-End Data Pipeline Implementation Using Azure Data Factory and Azure Blob Storage

Problem Statement

Build a complete pipeline that reads a CSV file from Azure Blob Storage and processes it using Azure Data Factory.

Objective

To create an end-to-end data pipeline that reads a CSV file from Azure Blob Storage, validates file metadata, copies the file to a new location using Azure Data Factory, and verifies successful execution.

Architecture

Source Blob Container
↓
Get Metadata Activity
↓
Copy Data Activity
↓
Destination Blob Container

Step 1: Create Resource Group

Procedure

1. Logged into Azure Portal using Azure for Students account.
2. Opened Resource Groups.
3. Clicked Create Resource Group.
4. Entered the following details:

Resource Group Name: rg-adf-mini-project

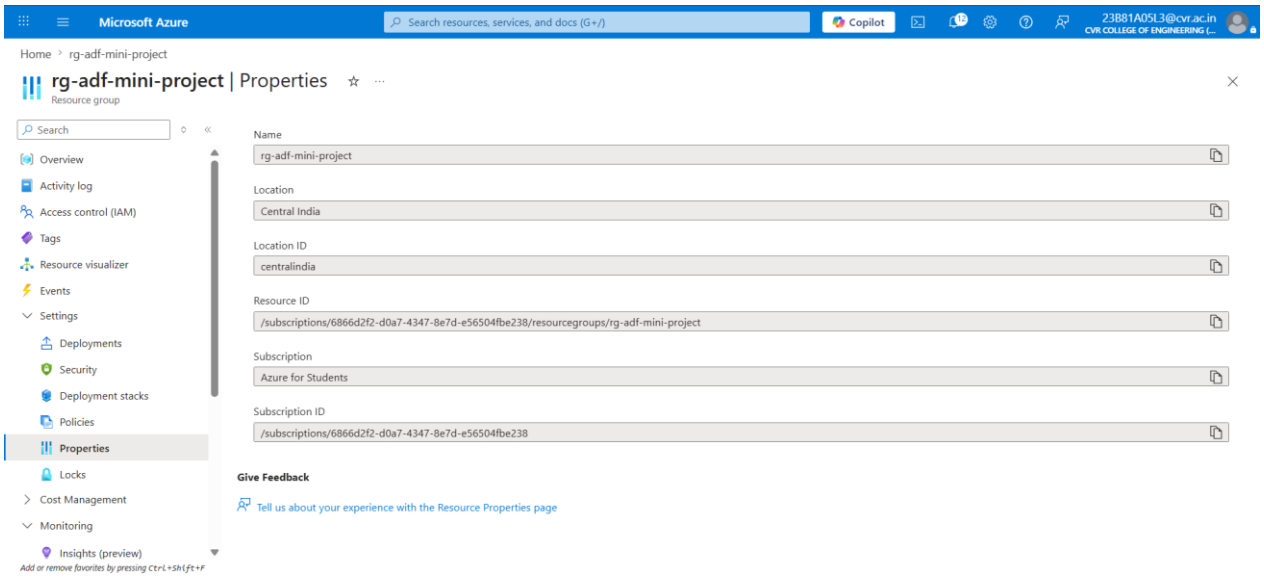
Region: Central India

5. Clicked Review + Create.
6. Successfully created the Resource Group.

Output

Resource Group was successfully created and used to organize all project resources.

Screenshot 1



Caption: Resource Group Creation

Step 2: Create Storage Account

Procedure

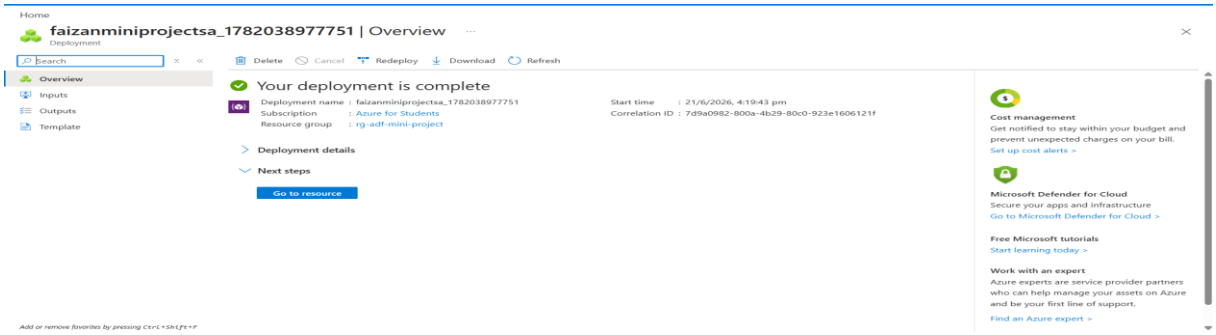
1. Opened Storage Accounts.
2. Clicked Create.
3. Selected the Resource Group.
4. Entered the Storage Account Name.

5. Selected Standard Performance.
6. Selected Locally Redundant Storage (LRS).
7. Created the Storage Account successfully.

Output

Storage Account was successfully created.

Screenshot 2



Caption: Storage Account Overview

Step 3: Create Blob Containers

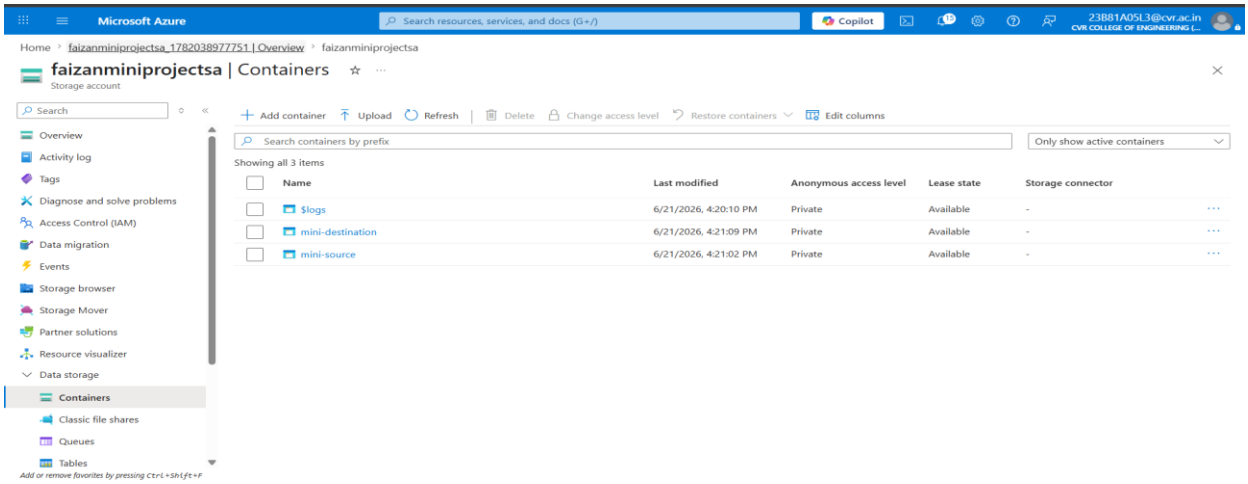
Procedure

1. Opened the Storage Account.
2. Navigated to Containers.
3. Created the following containers:
    - mini-source
    - mini-destination

Output

Both source and destination containers were created successfully.

Screenshot 3



Caption: Source and Destination Containers

Step 4: Upload CSV File

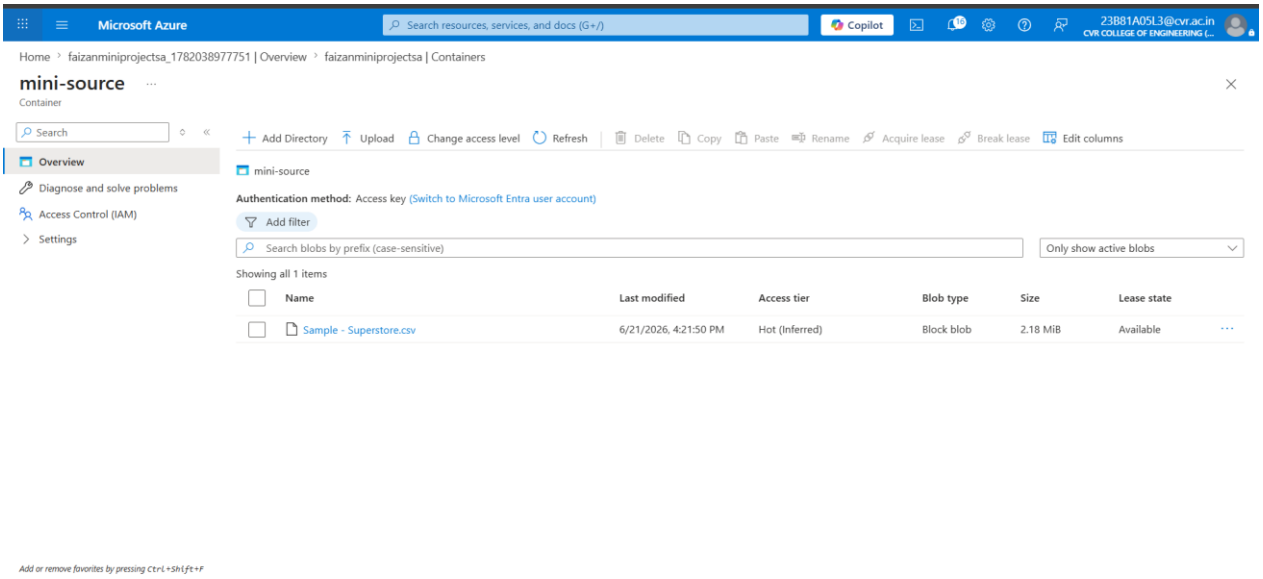
Procedure

1. Opened the mini-source container.
2. Clicked Upload.
3. Selected Sample-Superstore.csv.
4. Uploaded the file successfully.

Output

CSV file was successfully uploaded into the source container.

Screenshot 4



Caption: Source Container with Uploaded CSV File

Step 5: Create Azure Data Factory

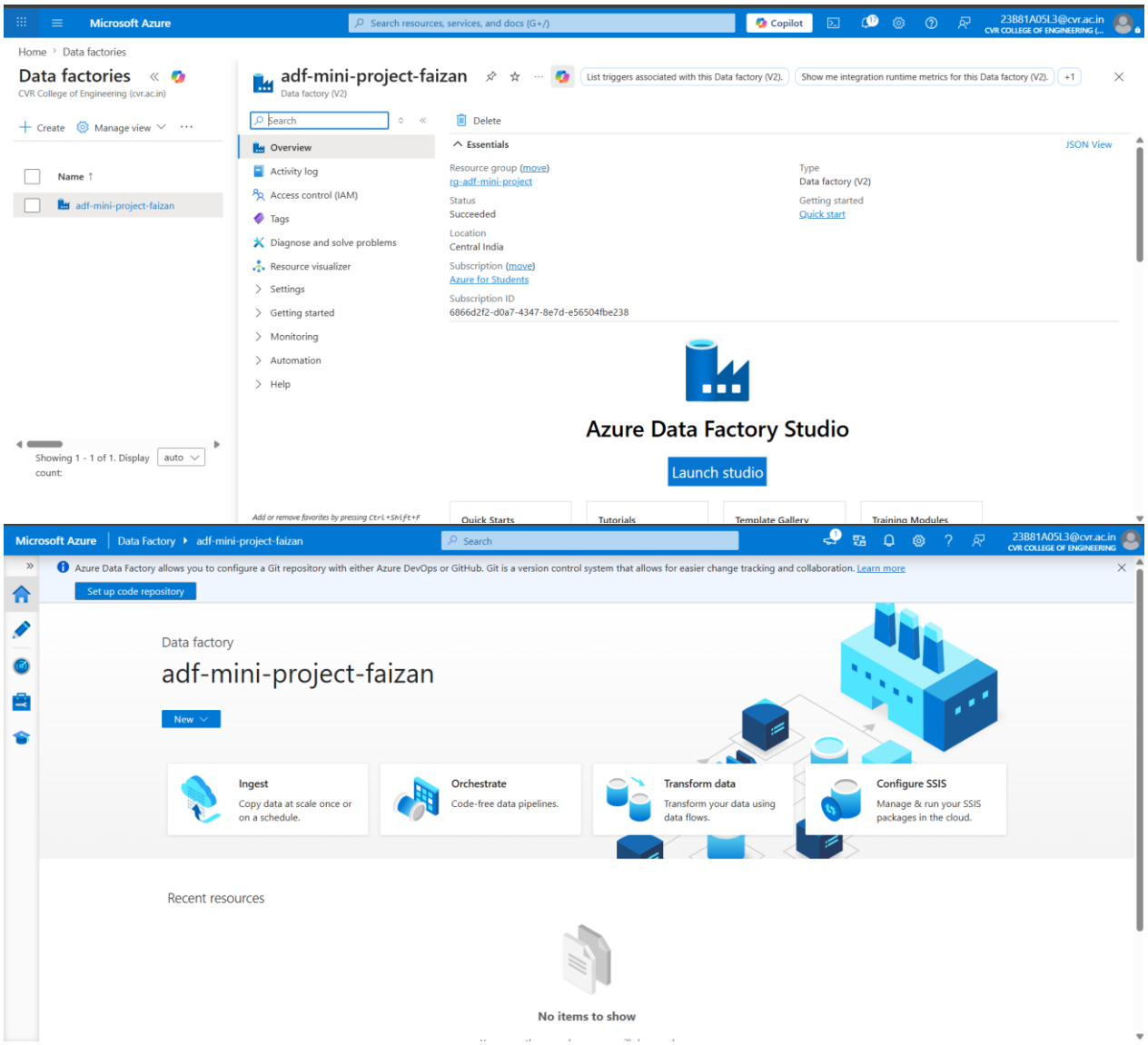
Procedure

1. Opened Azure Data Factory service.
2. Clicked Create.
3. Selected the Resource Group.
4. Entered the Data Factory Name.
5. Created the Azure Data Factory instance.
6. Opened ADF Studio.

Output

Azure Data Factory was successfully created and launched.

Screenshot 5



Caption: Azure Data Factory Overview

Step 6: Create Linked Service

Procedure

1. Opened ADF Studio.
2. Navigated to Manage → Linked Services.
3. Clicked New.
4. Selected Azure Blob Storage.
5. Configured the following:

Name: LS_Mini_BlobStorage

Authentication Type: Account Key

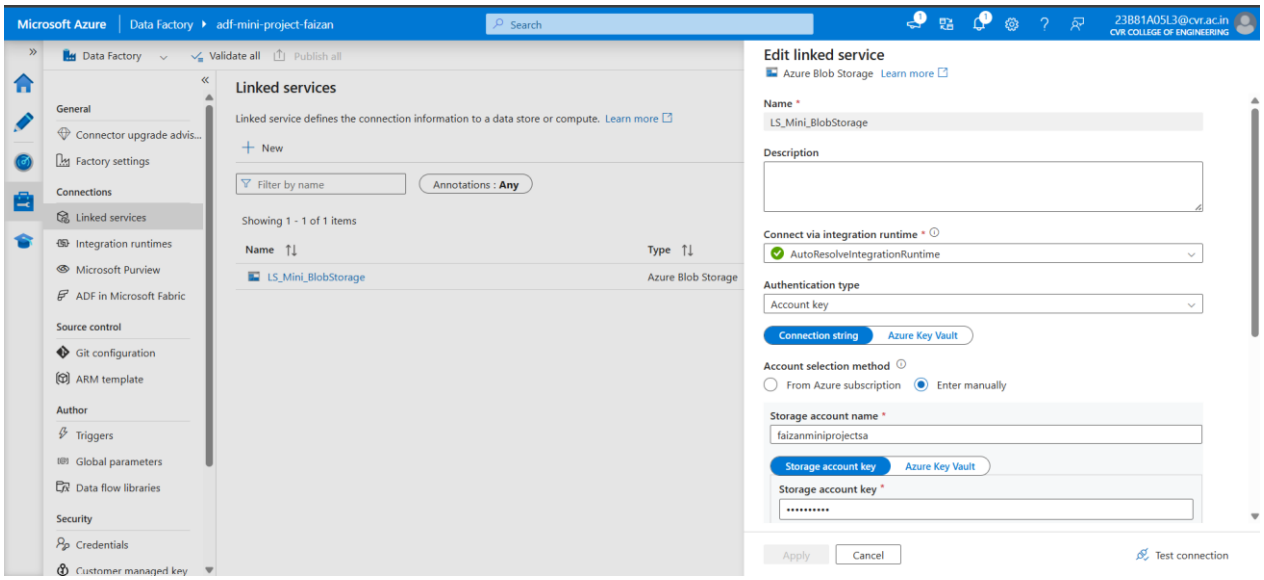
Storage Account: Selected Storage Account

6. Tested the connection.
7. Created the Linked Service successfully.

Output

Azure Blob Storage was successfully connected to Azure Data Factory.

Screenshot 6



Caption: Linked Service Configuration

Step 7: Create Source Dataset

Procedure

1. Navigated to Author → New Dataset.
2. Selected Azure Blob Storage.
3. Selected Delimited Text format.
4. Configured:

Dataset Name: DS_Mini_Source

Container: mini-source

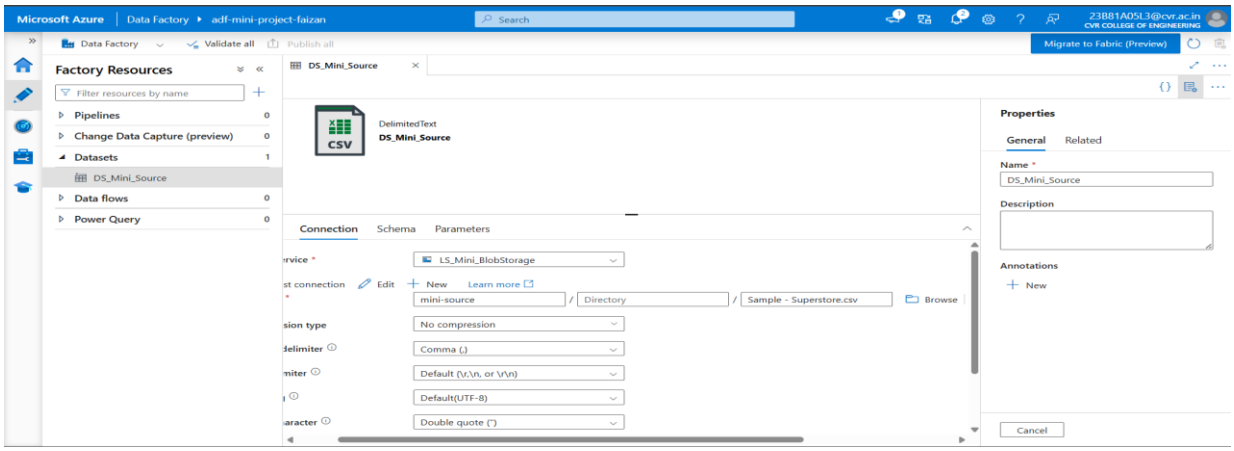
File Name: Sample-Superstore.csv

5. Saved the dataset.

Output

Source dataset was successfully created.

Screenshot 7



Caption: Source Dataset Configuration

Step 8: Create Destination Dataset

Procedure

1. Created another dataset.
2. Selected Azure Blob Storage.
3. Selected Delimited Text format.
4. Configured:

Dataset Name: DS_Mini_Destination

Container: mini-destination

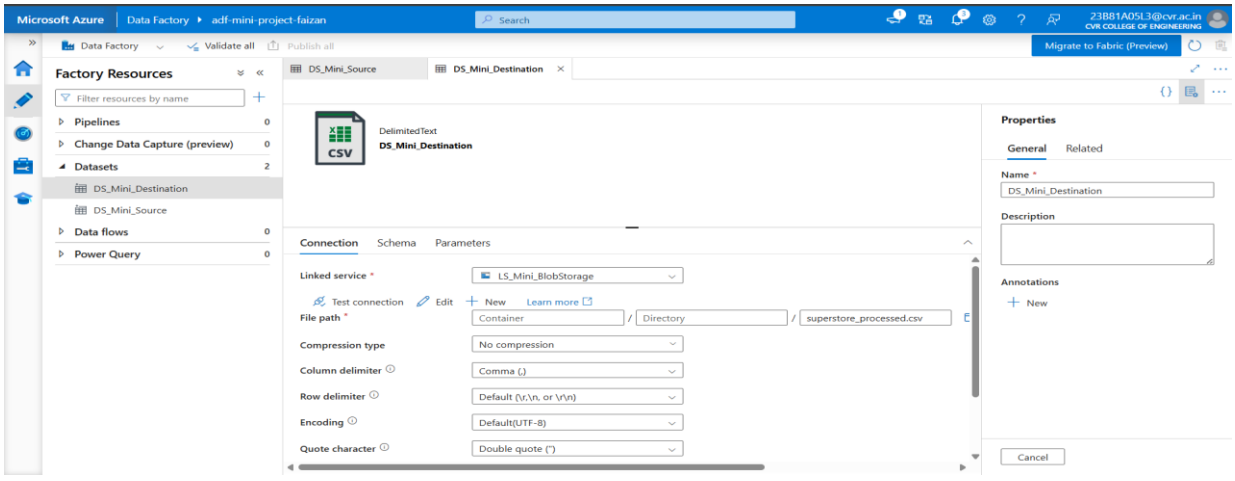
File Name: superstore_processed.csv

5. Saved the dataset.

Output

Destination dataset was successfully created.

Screenshot 8



Caption: Destination Dataset Configuration

Step 9: Create Pipeline

Procedure

1. Created a new pipeline.
2. Named the pipeline:

PL_Mini_Project

3. Added a Get Metadata activity.
4. Renamed the activity:

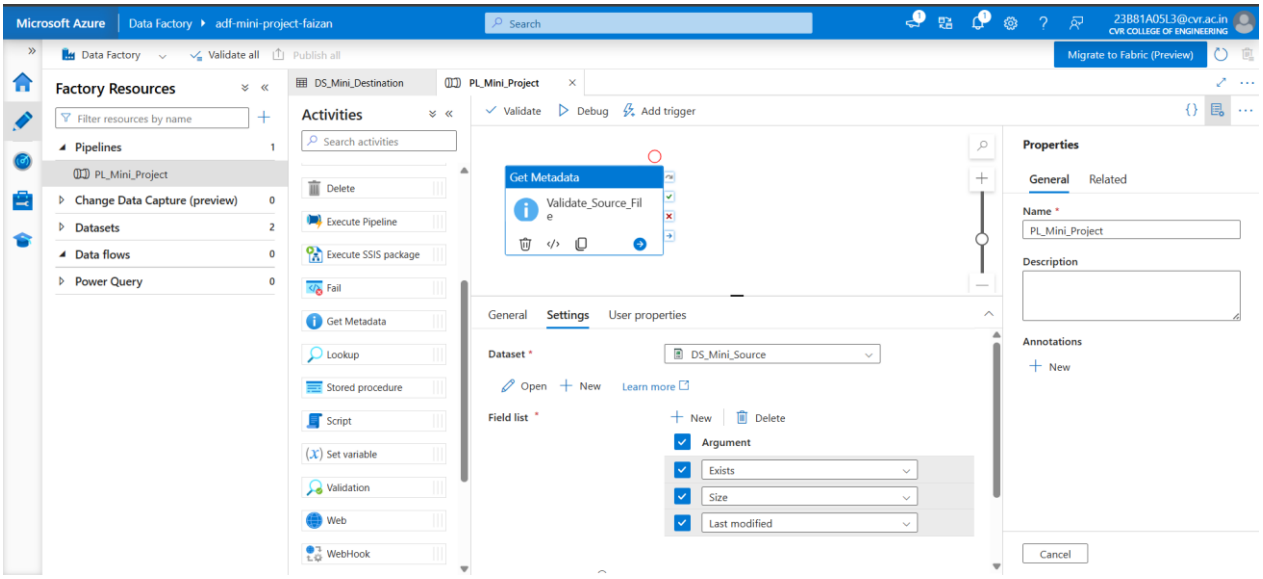
Validate_Source_File

5. Selected DS_Mini_Source as the dataset.
6. Configured the following metadata fields:
    - Exists
    - Size
    - Last Modified

Output

Metadata validation activity was successfully configured.

Screenshot 9



Caption: Get Metadata Activity Configuration

Step 10: Configure Copy Data Activity

Procedure

1. Added a Copy Data activity.
2. Renamed the activity:

Copy_Superstore_File

3. Connected the activity after Get Metadata.
4. Configured Source:

Dataset: DS_Mini_Source

5. Configured Sink:

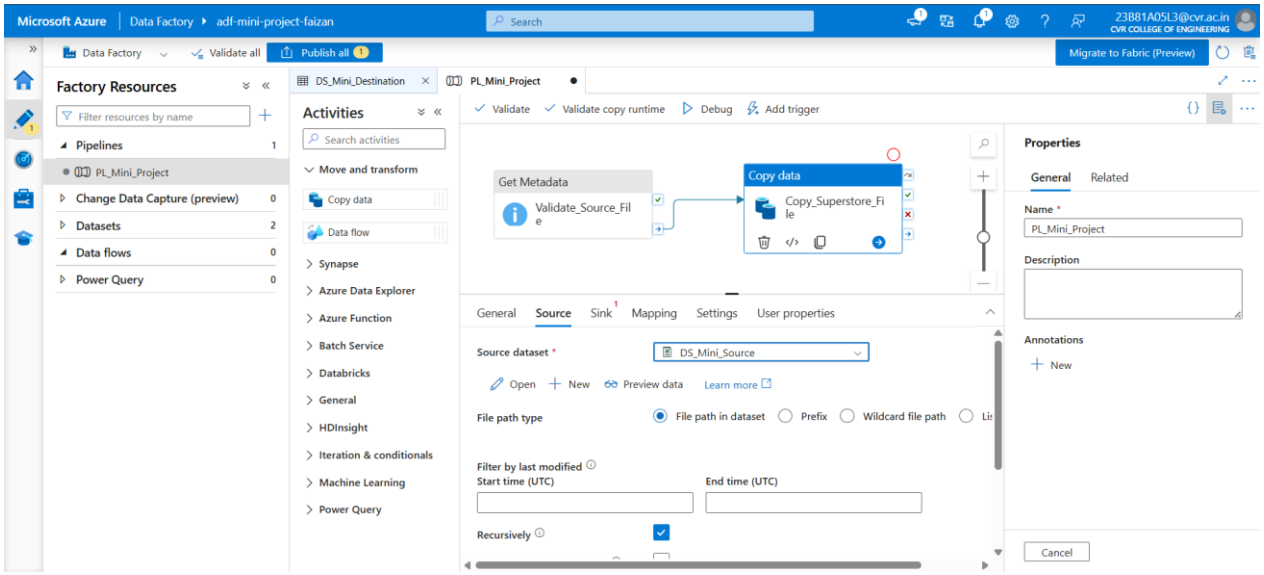
Dataset: DS_Mini_Destination

6. Validated the pipeline successfully.

Output

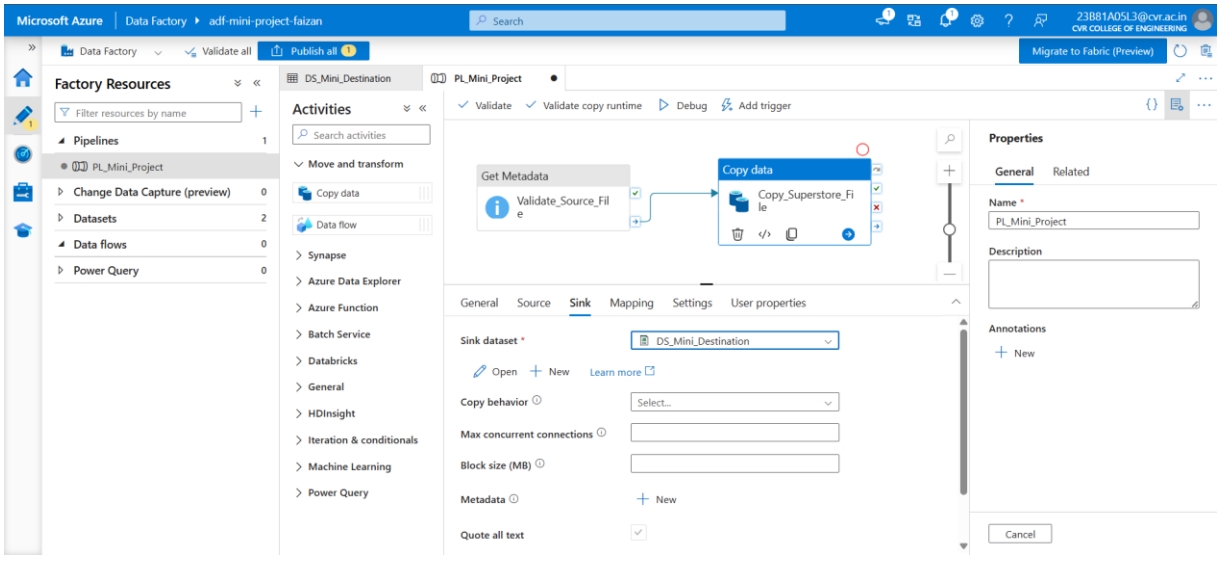
Copy Data activity was successfully configured.

Screenshot 10



Caption: Copy Data Source Configuration

Screenshot 11



Caption: Copy Data Sink Configuration

Step 11: Pipeline Design

Procedure

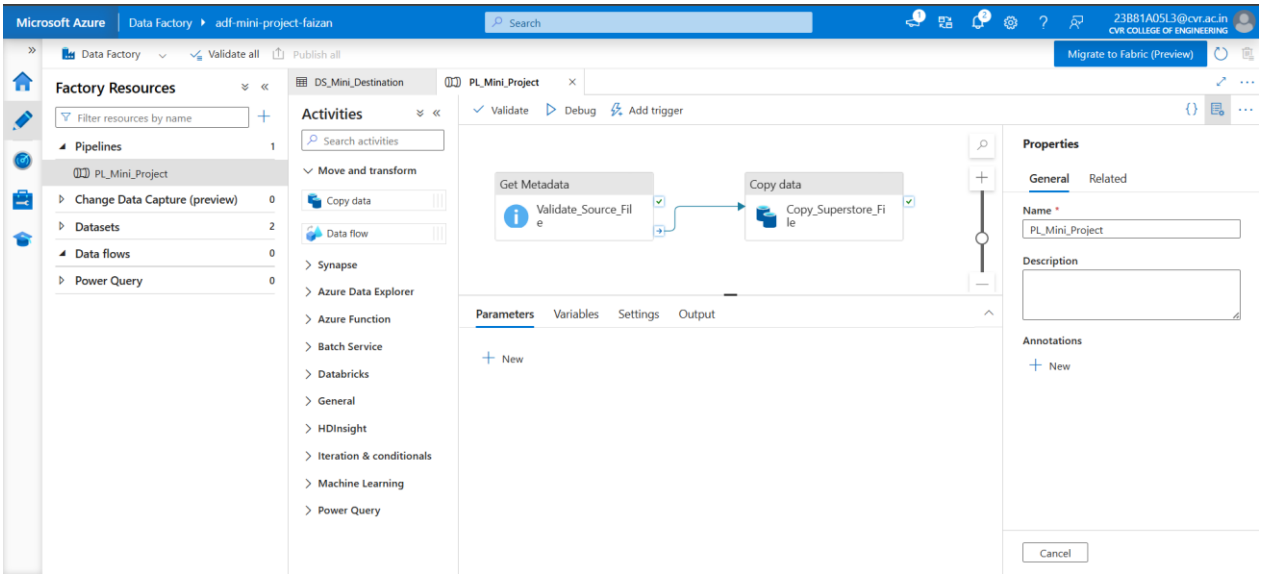
The final pipeline structure was:

Validate_Source_File
↓
Copy_Superstore_File

Output

Pipeline design completed successfully.

Screenshot 12



Caption: Complete Pipeline Design

Step 12: Execute Pipeline

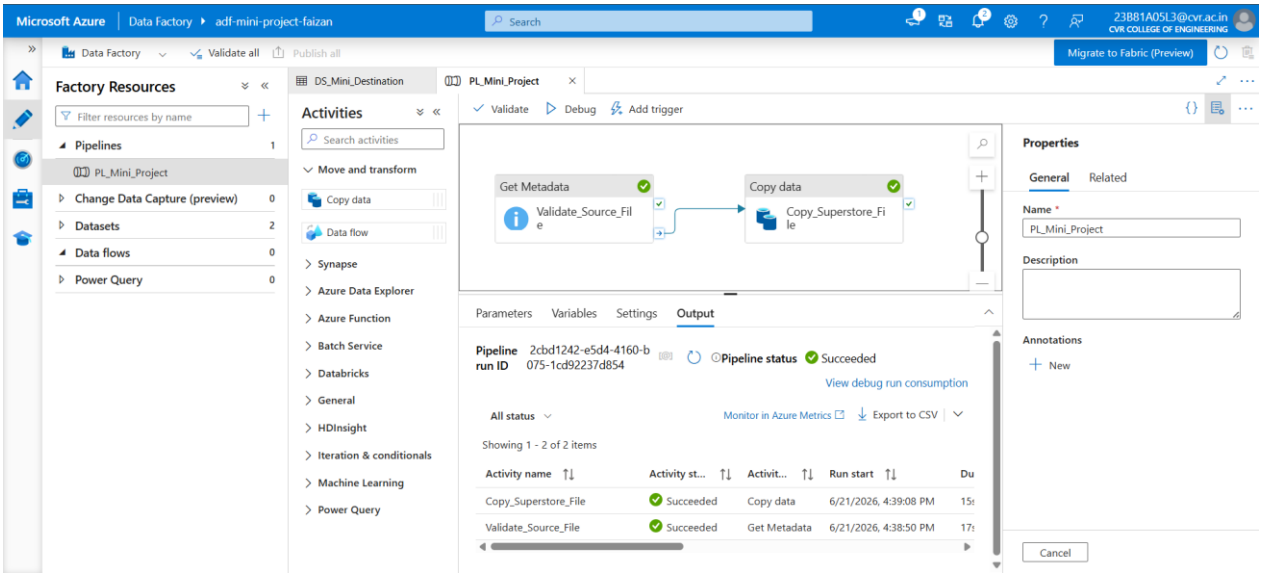
Procedure

1. Clicked Validate.
2. Confirmed that no validation errors existed.
3. Clicked Publish All.
4. Executed the pipeline using Debug.
5. Monitored execution status.

Output

Pipeline execution completed successfully.

Screenshot 13



Caption: Successful Pipeline Execution

Step 13: Verify Output

Procedure

1. Opened the mini-destination container.
2. Verified the copied file.

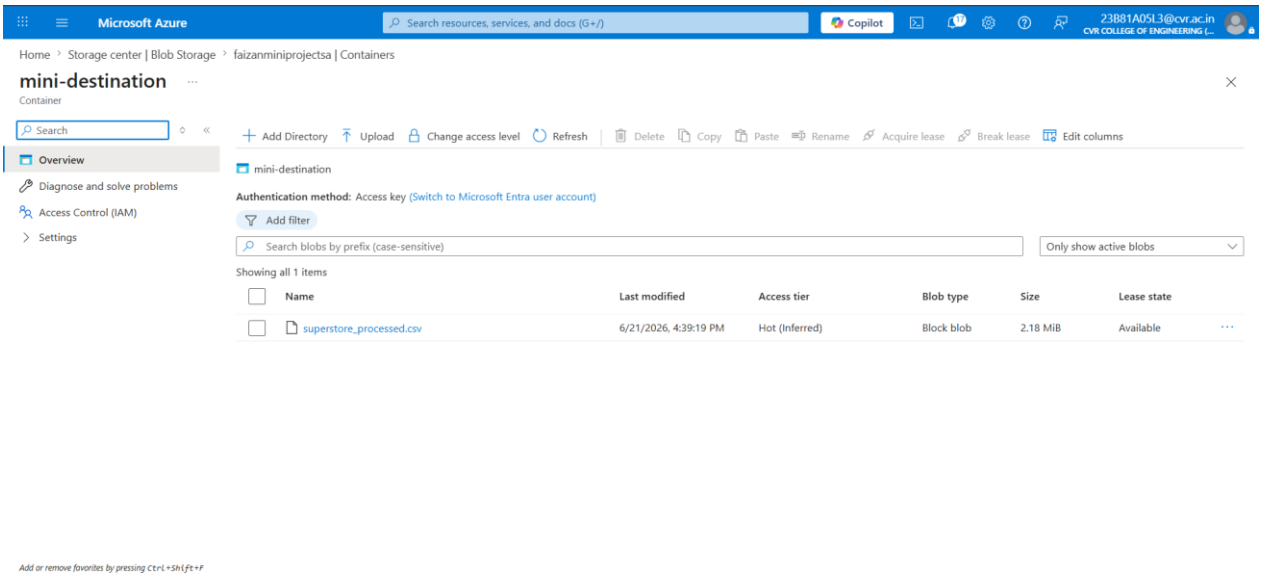
Output

The file was successfully copied to the destination container.

Output File:

superstore_processed.csv

Screenshot 14



Caption: Output File in Destination Container

Results

The pipeline successfully performed the following operations:

- Read the CSV file from Azure Blob Storage
- Retrieved metadata including file existence, size, and last modified timestamp
- Copied the file to a new destination container
- Executed successfully without errors
- Verified the output file in the destination container

Overall Observation

During this assignment, I explored various Azure cloud services and gained practical experience in creating and managing cloud resources. I successfully created Resource Groups, Storage Accounts, Blob Containers, and Azure Data Factory instances. I configured Linked Services and Datasets to establish connectivity between Azure Data Factory and Azure Blob Storage.

I implemented data pipelines using Get Metadata, ForEach, and Copy Data activities to automate file processing and data movement. I also learned how metadata validation helps verify files before processing and how Azure IAM roles control access to cloud resources. By monitoring pipeline execution and validating outputs, I was able to understand the complete workflow of data integration and orchestration in Azure.

The assignment provided hands-on exposure to cloud-based data engineering concepts and demonstrated how Azure Data Factory can be used to build scalable and automated data pipelines.

Conclusion

The objectives of the assignment were successfully achieved. Azure resources were created and configured correctly, and an end-to-end data pipeline was implemented using Azure Blob Storage and Azure Data Factory. The pipeline successfully retrieved metadata, processed CSV files, and copied data from the source location to the destination location.

The successful execution of both the assignment pipeline and the mini project confirmed that Azure Data Factory can efficiently orchestrate data movement and processing tasks. Through this assignment, I gained practical knowledge of Azure cloud services, storage management, access control, pipeline development, and monitoring. This experience strengthened my understanding of cloud-based data engineering workflows and their real-world applications.